



Generative Adversarial Networks (GANs)

Prof. Anil Singh Parihar

Department of Computer Science & Engineering

Delhi Technological University, Delhi



Generative Models

Autoregressive Models directly maximize likelihood of training data:

$$p_{\theta}(x) = \prod_{i=1}^N p_{\theta}(x_i | x_1, x_2, \dots, x_{i-1})$$

Variational Autoencoders introduce a latent z , and maximize a lower bound:

$$p_{\theta}(x) = \int p_{\theta}(x|z)p(z) dz \geq \mathbb{E}_{z \sim q_{\phi}(z/x)} [\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x) \| p(z))$$

Generative Adversarial Networks give up on modeling $p(x)$, but allow us to draw samples from $p(x)$.

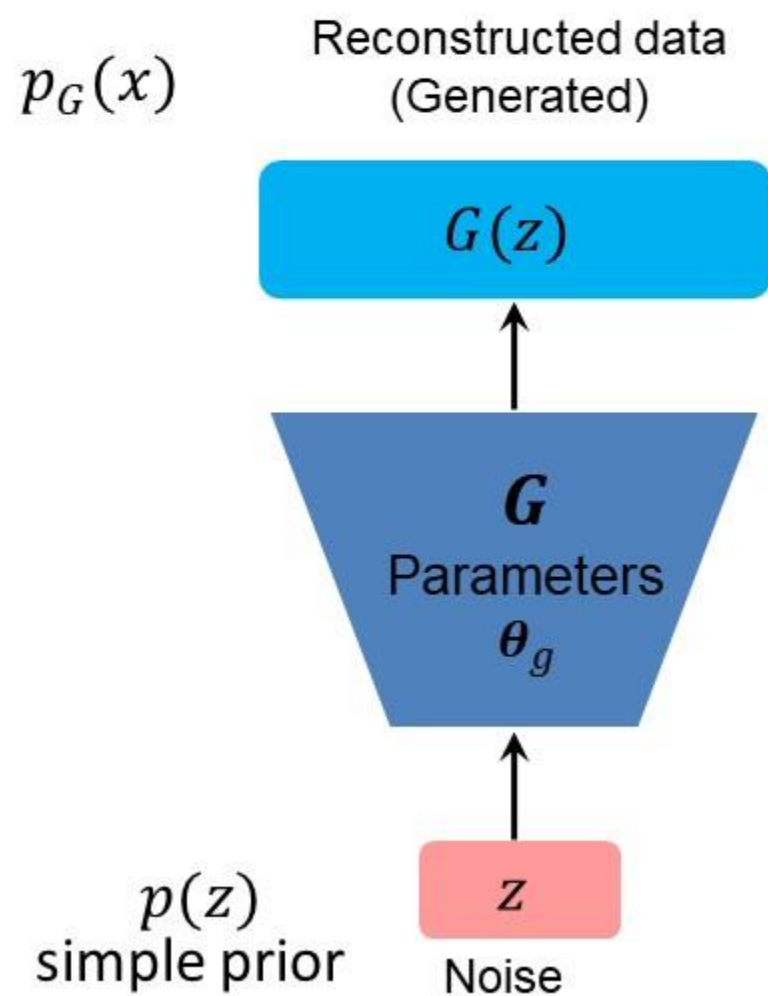


Generative Adversarial Networks

- **Setup:** Assume we have data x_i drawn from distribution $p_{data}(x)$.
- $p_{data}(x)$ is actual distribution of data, but we can't find it,
- So, we want sample from it i.e. we generate sample from the data.

x $p_{data}(x)$
Original data

Generative Adversarial Networks



Idea: Introduce a latent variable z with simple prior $p(z)$. Sample $z \sim p(z)$ and pass to a **Generator Network** $x = G(z)$

Then x is a sample from the **Generator distribution** p_G .

Goal: $p_G = p_{\text{data}}$!

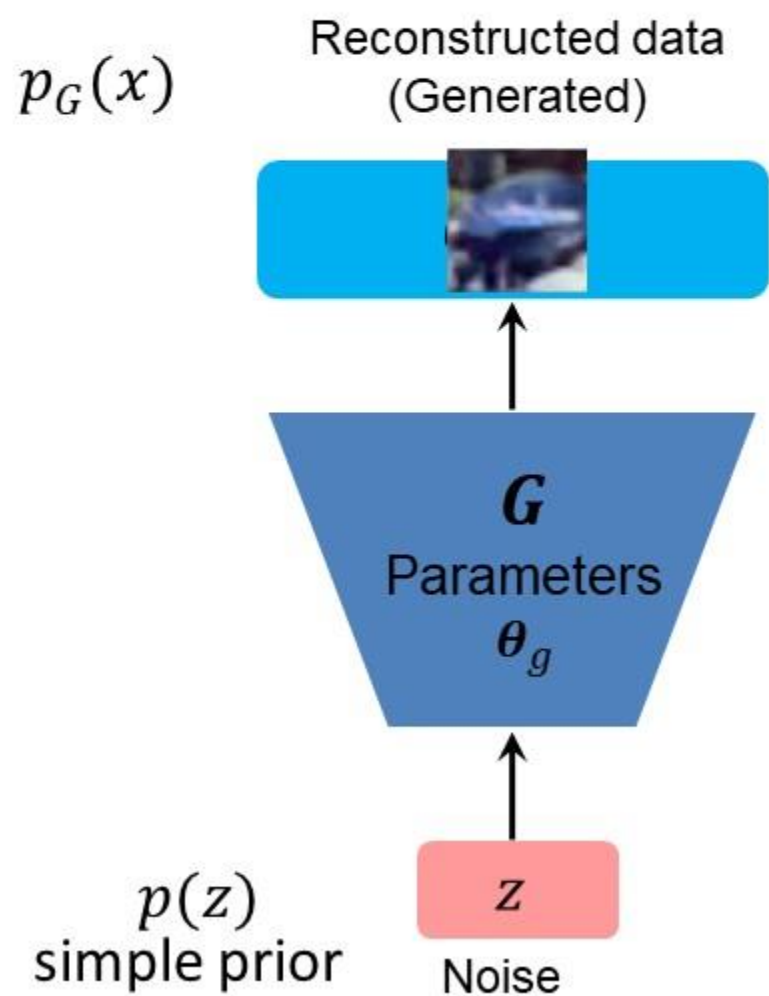
x
Original data

$p_{\text{data}}(x)$

We can not find
 $p_{\text{data}}(x)$

Generative Adversarial Networks

Train **Generator Network G** to convert z into fake data x sampled from p_G

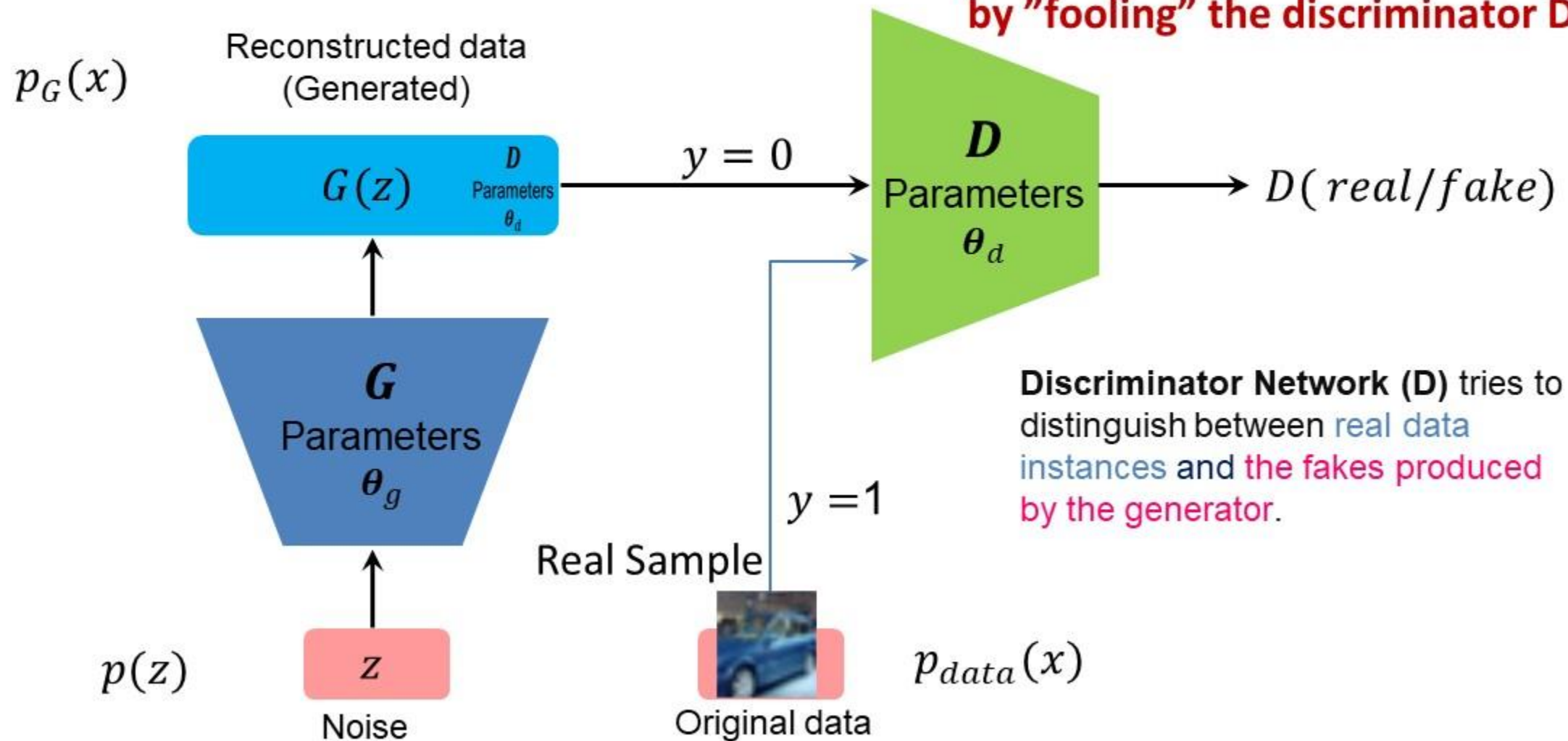


Generator network (G) attempts to generate new data instances that resemble the real data.

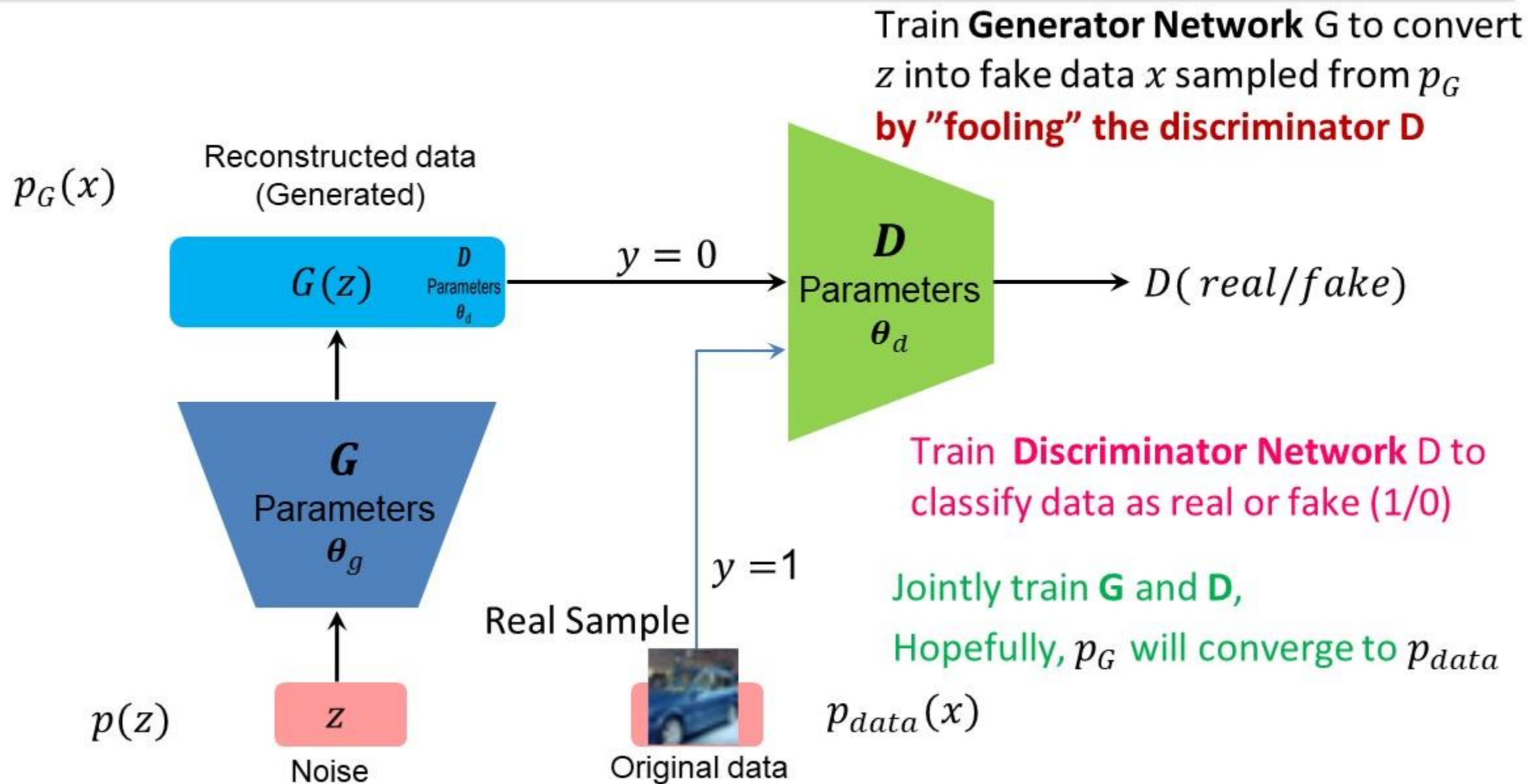
x Original data $p_{data}(x)$ **We can not find** $p_{data}(x)$

Generative Adversarial Networks

Train **Generator Network G** to convert z into fake data x sampled from p_G by "fooling" the discriminator **D**



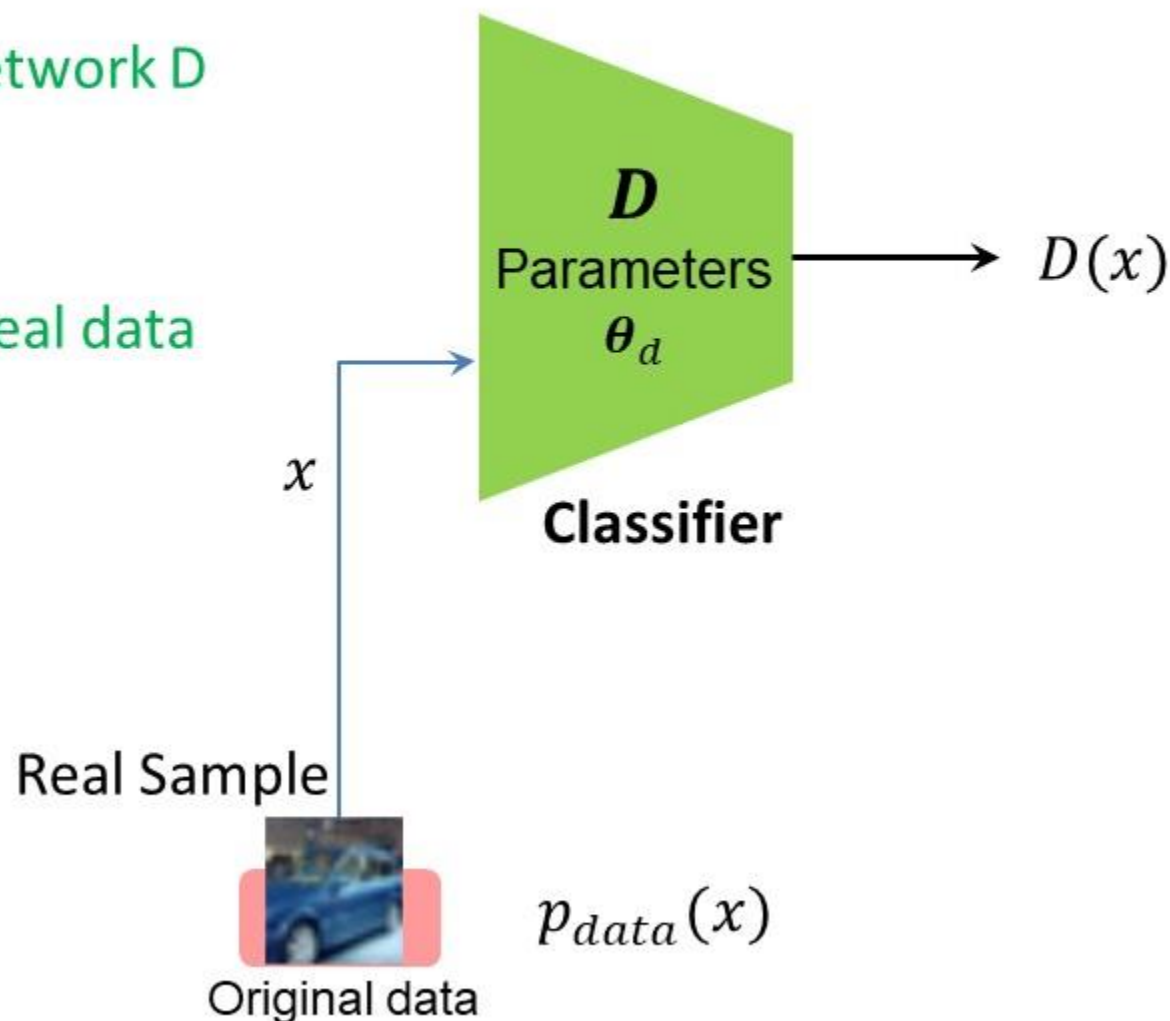
Generative Adversarial Networks



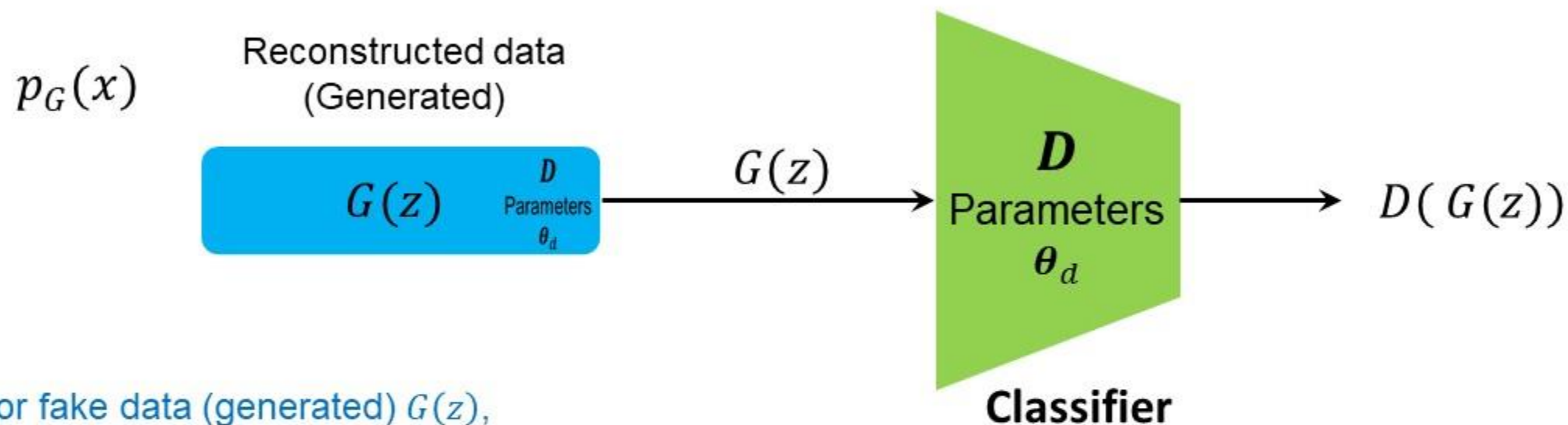
Generative Adversarial Networks

For real samples, discriminator network D tries to keep value of $D(x)$ maximum i.e. close to 1.

$D(x)$ is probability of x being real data



Generative Adversarial Networks



For fake data (generated) $G(z)$,
The Discriminator network D tries to keep value of $D(G(z))$ minimum i.e. close to 0.

Note: $D(x)$ is probability of x being real data,
thus for $x = G(z)$ (fake data)
 $D(G(z))$ should be close to zero

Since $0 \leq D(G(z)) \leq 1$

Minimizing $D(G(z))$ is same as

Maximizing $(1 - D(G(z)))$



Generative Adversarial Networks

Jointly train generator G and discriminator D with a **minimax game**

The training of GANs can be formulated as a min-max game with the following value function $V(D, G)$

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$

Does it look like similar to
binary cross entropy?



Generative Adversarial Networks

Binary cross entropy loss (for single observation):

$$L = -[(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))]$$

For real sample i.e. $y = 1$, $\hat{y} = D(x)$

$$L = -\log D(x)$$

For fake sample i.e. $y = 0$, $\hat{y} = D(G(z))$

$$L = -[\log(1 - D(G(z)))]$$



Generative Adversarial Networks

Binary cross entropy loss (for single observation):

$$L = -[(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))]$$

For real sample i.e. $y = 1, \hat{y} = D(x)$ $L = -\log D(x)$

For fake sample i.e. $y = 0, \hat{y} = D(G(z))$ $L = -[\log(1 - D(G(z)))]$

Adding both, taking negative of it (i.e. we have to now maximize L)

$$L = \log D(x) + \log(1 - D(G(z)))$$



Generative Adversarial Networks

Negative Binary cross entropy loss (for single observation):

$$L = \log D(x) + \log(1 - D(G(z)))$$

For N number of observation, we average of expectation of L

$$\mathbb{E}(L) = \mathbb{E}(\log D(x)) + \mathbb{E}(\log(1 - D(G(z))))$$

$$\mathbb{E}(L) = \sum p_{data}(x) \log D(x) + \sum p(z) \log(1 - D(G(z)))$$

$$\mathbb{E}(L) = \int p_{data}(x) \log D(x) dx + \int p(z) \log(1 - D(G(z))) dz$$



Generative Adversarial Networks

Negative Binary cross entropy loss (for single observation):

$$L = \log D(x) + \log(1 - D(G(z)))$$

For N number of observation, we average of expectation of L

$$\mathbb{E}(L) = \mathbb{E}(\log D(x)) + \mathbb{E}(\log(1 - D(G(z))))$$

$$\mathbb{E}(L) = \int p_{data}(x) \log D(x) dx + \int p(z) \log(1 - D(G(z))) dz$$

$$\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]$$



Generative Adversarial Networks

Jointly train generator G and discriminator D with a **minimax game**

The training of GANs can be formulated as a min-max game with the following value function $V(D, G)$

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$

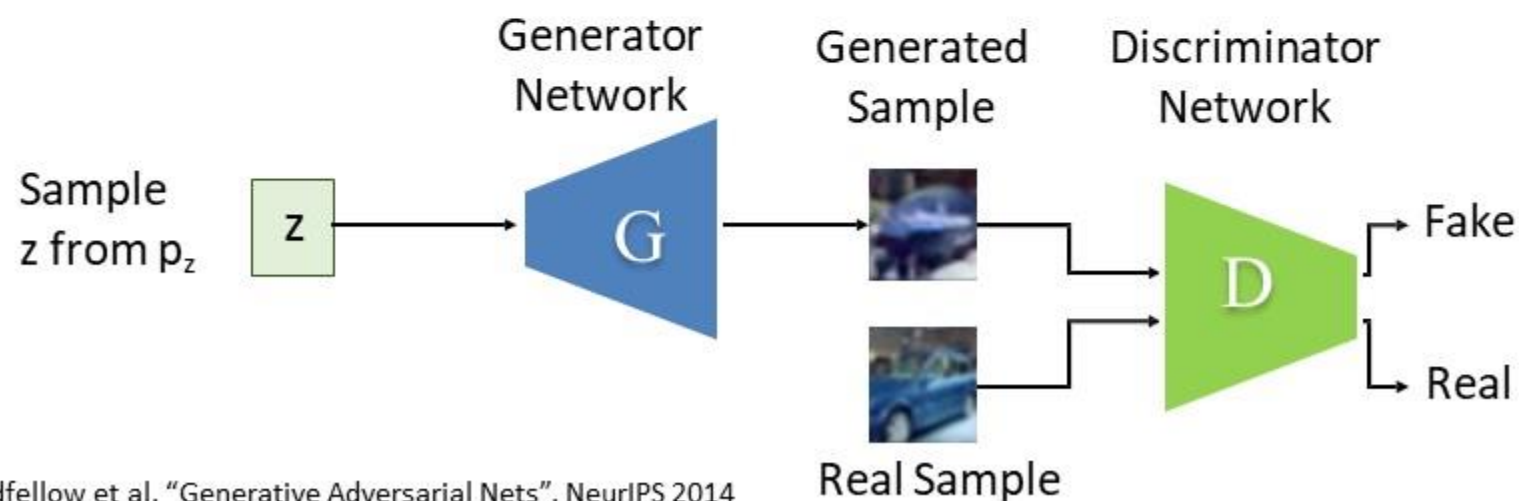
Generative Adversarial Networks

Jointly train generator G and discriminator D with a **minimax game**

Discriminator wants
 $D(x) = 1$ for real data

Discriminator wants
 $D(x) = 0$ for fake data

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$



Generator wants
 $D(x) = 1$ for fake data

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

Train G and D using alternating gradient updates

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$

$\mathbb{E}_{x \sim p_{data}} [\log D(x)]$: This term represents the discriminator's ability to recognize real data.

The discriminator aims to maximize this term, making it close to 1 (or maximizing the logarithm of the discriminator's output for real data).

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

Train G and D using alternating gradient updates

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

$\mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right]$: This term represents the discriminator's ability to recognize fake data generated by the generator.

The discriminator wants to maximize this term by minimizing $\mathbf{D}(\mathbf{G}(z))$ pushing its output toward 0 for fake data.

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

Train G and D using alternating gradient updates

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$

For the Discriminator: Maximize $\log D(x) + \log \left(1 - D(G(z)) \right)$ for given G.

For the Generator: Minimize $\log \left(1 - D(G(z)) \right)$, or equivalently, maximize $\log \left(D(G(z)) \right)$ to fool the discriminator into thinking the fake samples are real..

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

Train G and D using alternating gradient updates

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$
$$= \min_{\mathbf{G}} \max_{\mathbf{D}} \mathbf{V}(\mathbf{G}, \mathbf{D})$$

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

Train G and D using alternating gradient updates

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

$$= \min_{\mathbf{G}} \max_{\mathbf{D}} V(\mathbf{G}, \mathbf{D})$$

For t in $1, \dots T$:

1. (Update $\mathbf{D}$) $\mathbf{D} = \mathbf{D} + \alpha_{\mathbf{D}} \frac{\partial V}{\partial \mathbf{D}}$
2. (Update $\mathbf{G}$) $\mathbf{G} = \mathbf{G} - \alpha_{\mathbf{G}} \frac{\partial V}{\partial \mathbf{G}}$

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

Train G and D using alternating gradient updates

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

$$= \min_{\mathbf{G}} \max_{\mathbf{D}} V(\mathbf{G}, \mathbf{D})$$

We are not minimizing any overall loss! No training curves to look at!

For t in $1, \dots, T$:

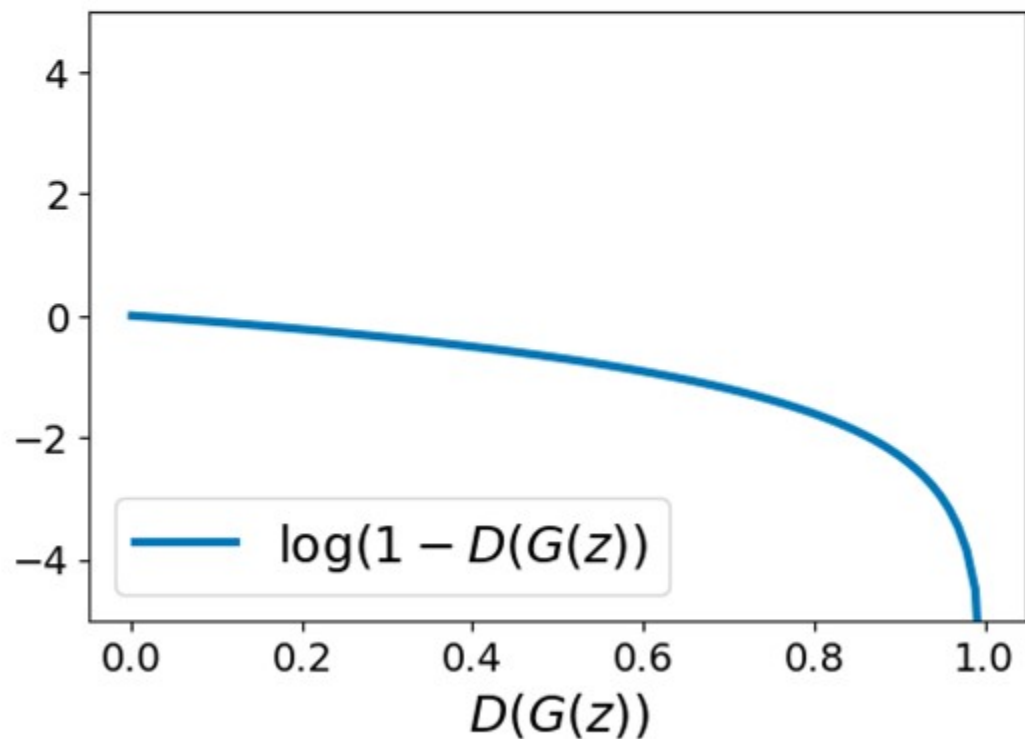
1. (Update $\mathbf{D}$) $\mathbf{D} = \mathbf{D} + \alpha_{\mathbf{D}} \frac{\partial V}{\partial \mathbf{D}}$
2. (Update $\mathbf{G}$) $\mathbf{G} = \mathbf{G} - \alpha_{\mathbf{G}} \frac{\partial V}{\partial \mathbf{G}}$

Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$

At start of training, generator is very bad and discriminator can easily tell apart real/fake, so $D(G(z))$ close to 0



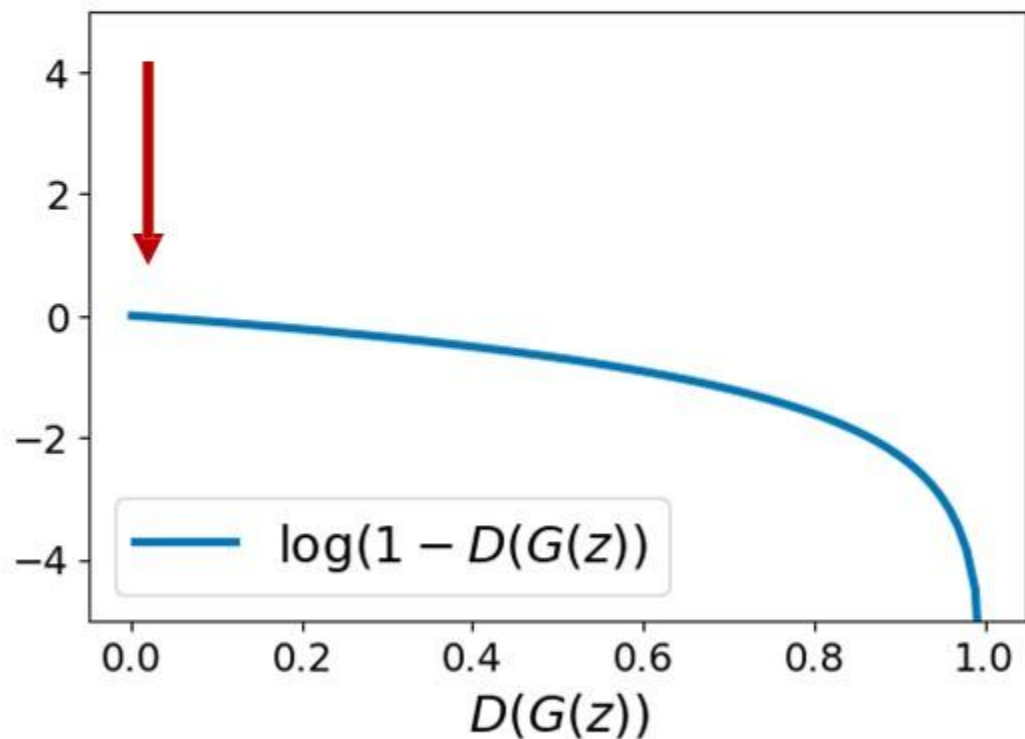
Generative Adversarial Networks: Training Objective

Jointly train generator G and discriminator D with a **minimax game**

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log (1 - D(G(z)))] \right)$$

At start of training, generator is very bad and discriminator can easily tell apart real/fake, so $D(G(z))$ close to 0

Problem: Vanishing gradients for G



Generative Adversarial Networks: Training Objective

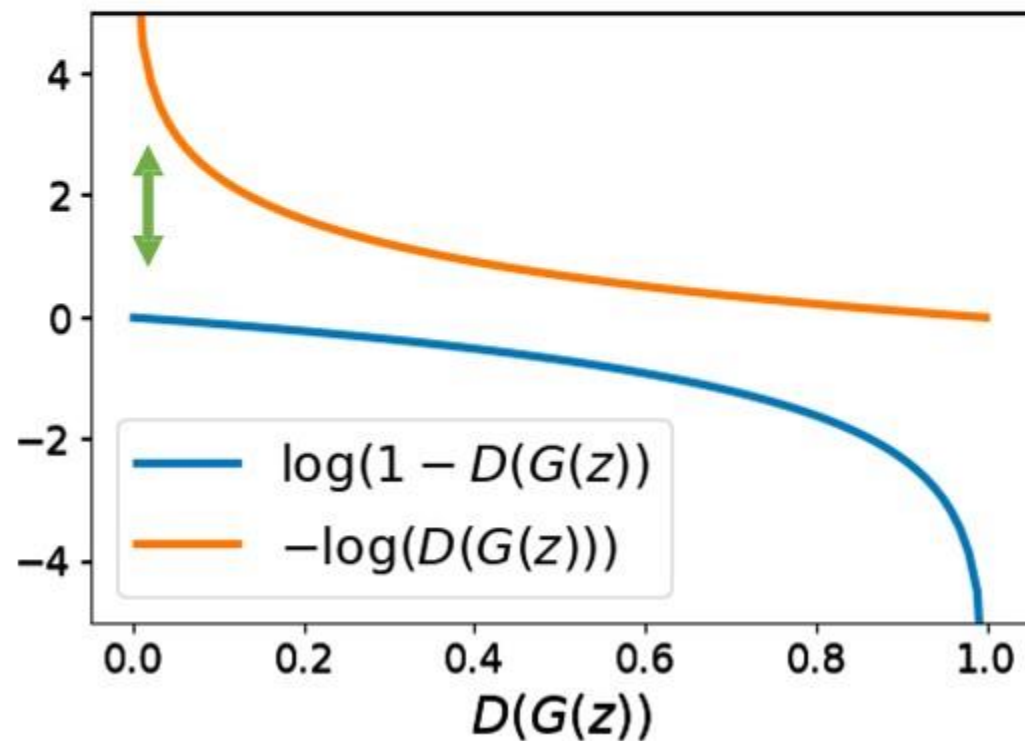
Jointly train generator G and discriminator D with a **minimax game**

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{x \sim p(z)} [\log (1 - D(G(x)))] \right)$$

At start of training, generator is very bad and discriminator can easily tell apart real/fake, so $D(G(z))$ close to 0

Problem: Vanishing gradients for G

Solution: Right now G is trained to minimize $\log(1 - D(G(z)))$. Instead, train G to minimize $-\log(D(G(z)))$. Then G gets strong gradients at start of training!



Generative Adversarial Networks: Optimality

Jointly train generator G and discriminator D with a **minimax game**

Why is this particular objective a good idea?

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$

This minimax game achieves its global minimum when $p_G = p_{data}$!

Generative Adversarial Networks: Optimality

$$\min_{\textcolor{red}{G}} \max_{\textcolor{blue}{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \textcolor{blue}{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \textcolor{blue}{D}(\textcolor{red}{G}(z)) \right) \right] \right)$$

(Our objective so far)

Generative Adversarial Networks: Optimality

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

$$= \min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{x \sim p_{\mathbf{G}}} [\log (1 - \mathbf{D}(x))] \right)$$

(Change of variables on second term)

Generative Adversarial Networks: Optimality

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

$$= \min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{x \sim p_{\mathbf{G}}} [\log (1 - \mathbf{D}(x))] \right)$$

$$= \min_{\mathbf{G}} \max_{\mathbf{D}} \int_X \left(p_{data} \log \mathbf{D}(x) + p_{\mathbf{G}}(x) \log (1 - \mathbf{D}(x)) \right) dx$$

(Definition of expectation)

Generative Adversarial Networks: Optimality

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

$$= \min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{x \sim p_{\mathbf{G}}} [\log (1 - \mathbf{D}(x))] \right)$$

$$= \min_{\mathbf{G}} \int_X \max_{\mathbf{D}} \left(p_{data} \log \mathbf{D}(x) + p_{\mathbf{G}}(x) \log (1 - \mathbf{D}(x)) \right) dx$$

(Push $\max_{\mathbf{D}}$ inside integral)

Generative Adversarial Networks: Optimality

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

$$= \min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{x \sim p_{\mathbf{G}}} [\log (1 - \mathbf{D}(x))] \right)$$

$$= \min_{\mathbf{G}} \int_X \max_{\mathbf{D}} \left(\underbrace{p_{data} \log \mathbf{D}(x) + p_{\mathbf{G}}(x) \log (1 - \mathbf{D}(x))}_{\text{(Let for fixed G, we want to maximize it)}} \right) dx$$

(Let for fixed G, we want to maximize it)

Generative Adversarial Networks: Optimality

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

$$= \min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{x \sim p_{\mathbf{G}}} [\log (1 - \mathbf{D}(x))] \right)$$

$$= \min_{\mathbf{G}} \int_X \max_{\mathbf{D}} \left(p_{data} \log \mathbf{D}(x) + p_{\mathbf{G}}(x) \log (1 - \mathbf{D}(x)) \right) dx$$

$$f(y) = \underbrace{a \log y + b \log (1 - y)}$$

(maximize it by varying $y = \mathbf{D}(x)$)

Generative Adversarial Networks: Optimality

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

$$= \min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{x \sim p_{\mathbf{G}}} [\log (1 - \mathbf{D}(x))] \right)$$

$$= \min_{\mathbf{G}} \int_X \max_{\mathbf{D}} \left(p_{data} \log \mathbf{D}(x) + p_{\mathbf{G}}(x) \log (1 - \mathbf{D}(x)) \right) dx$$

$$f(y) = a \log y + b \log (1 - y)$$

$$f'(y) = \frac{a}{y} + \frac{b}{1 - y}$$

Generative Adversarial Networks: Optimality

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$

$$= \min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{x \sim p_G} [\log (1 - D(x))] \right)$$

$$= \min_G \int_X \max_D \left(p_{data} \log D(x) + p_G(x) \log (1 - D(x)) \right) dx$$

$$f(y) = a \log y + b \log (1 - y) \quad f'(y) = 0 \Leftrightarrow y = \frac{a}{a+b} \text{ (Local Max)}$$

$$f'(y) = \frac{a}{y} + \frac{b}{1-y}$$

Generative Adversarial Networks: Optimality

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$

$$= \min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{x \sim p_G} [\log (1 - D(x))] \right)$$

$$= \min_G \int_X \max_D \left(p_{data} \log D(x) + p_G(x) \log (1 - D(x)) \right) dx$$

$$f(y) = a \log y + b \log (1 - y) \quad f'(y) = 0 \Leftrightarrow y = \frac{a}{a+b} \text{ (Local Max)}$$

$$f'(y) = \frac{a}{y} + \frac{b}{1-y}$$

$$\text{Optimal Discriminator: } D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$$

Generative Adversarial Networks: Optimality

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

$$= \min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{x \sim p_{\mathbf{G}}} [\log (1 - \mathbf{D}(x))] \right)$$

$$= \min_{\mathbf{G}} \int_X \max_{\mathbf{D}} \left(p_{data} \log \mathbf{D}(x) + p_{\mathbf{G}}(x) \log (1 - \mathbf{D}(x)) \right) dx$$

$$\text{Optimal Discriminator: } \mathbf{D}_{\mathbf{G}}^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)}$$

Generative Adversarial Networks: Optimality

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

$$= \min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{x \sim p_{\mathbf{G}}} [\log (1 - \mathbf{D}(x))] \right)$$

$$= \min_{\mathbf{G}} \int_X \left(p_{data} \log D_{\mathbf{G}}^*(x) + p_{\mathbf{G}}(x) \log (1 - D_{\mathbf{G}}^*(x)) \right) dx$$

$$\text{Optimal Discriminator: } D_{\mathbf{G}}^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)}$$

Generative Adversarial Networks: Optimality

$$\min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right)$$

$$= \min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{x \sim p_{\mathbf{G}}} [\log (1 - \mathbf{D}(x))] \right)$$

$$= \min_{\mathbf{G}} \int_X \left(p_{data} \log D_{\mathbf{G}}^*(x) + p_{\mathbf{G}}(x) \log (1 - D_{\mathbf{G}}^*(x)) \right) dx$$

$$= \min_{\mathbf{G}} \int_X \left(p_{data} \log \frac{p_{data}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} + p_{\mathbf{G}}(x) \log \left(\frac{p_{\mathbf{G}}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} \right) \right) dx$$

$$\text{Optimal Discriminator: } D_{\mathbf{G}}^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)}$$

Generative Adversarial Networks: Optimality

$$\begin{aligned} & \min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right) \\ &= \min_{\mathbf{G}} \int_X \left(p_{data} \log \frac{p_{data}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} p_{\mathbf{G}}(x) \log \left(\frac{p_{\mathbf{G}}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} \right) \right) dx \end{aligned}$$

Generative Adversarial Networks: Optimality

$$\begin{aligned} & \min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right) \\ &= \min_{\mathbf{G}} \int_X \left(p_{data} \log \frac{p_{data}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} p_{\mathbf{G}}(x) \log \left(\frac{p_{\mathbf{G}}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} \right) \right) dx \\ &= \min_{\mathbf{G}} \left(\mathbb{E}_{x \sim p_{data}} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} \right] + \mathbb{E}_{x \sim p_{\mathbf{G}}} \left[\log \frac{p_{\mathbf{G}}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} \right] \right) \end{aligned}$$

(Definition of expectation)

Generative Adversarial Networks: Optimality

$$\begin{aligned} & \min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right) \\ &= \min_{\mathbf{G}} \int_X \left(p_{data} \log \frac{p_{data}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} + p_{\mathbf{G}}(x) \log \left(\frac{p_{\mathbf{G}}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} \right) \right) dx \\ &= \min_{\mathbf{G}} \left(\mathbb{E}_{x \sim p_{data}} \left[\log \frac{2}{2} \frac{p_{data}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} \right] + \mathbb{E}_{x \sim p_{\mathbf{G}}} \left[\log \frac{2}{2} \frac{p_{\mathbf{G}}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} \right] \right) \end{aligned}$$

(Multiply by a constant)

Generative Adversarial Networks: Optimality

$$\begin{aligned} & \min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right) \\ &= \min_{\mathbf{G}} \int_X \left(p_{data} \log \frac{p_{data}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} p_{\mathbf{G}}(x) \log \left(\frac{p_{\mathbf{G}}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} \right) \right) dx \\ &= \min_{\mathbf{G}} \left(\mathbb{E}_{x \sim p_{data}} \left[\log \frac{2}{2} \frac{p_{data}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} \right] + \mathbb{E}_{x \sim p_{\mathbf{G}}} \left[\log \frac{2}{2} \frac{p_{\mathbf{G}}(x)}{p_{data}(x) + p_{\mathbf{G}}(x)} \right] \right) \\ &= \min_{\mathbf{G}} \left(\mathbb{E}_{x \sim p_{data}} \left[\log \frac{p_{data}(x)}{\frac{p_{data}(x) + p_{\mathbf{G}}(x)}{2}} \right] + \mathbb{E}_{x \sim p_{\mathbf{G}}} \left[\log \frac{p_{\mathbf{G}}(x)}{\frac{p_{data}(x) + p_{\mathbf{G}}(x)}{2}} \right] - \log 4 \right) \end{aligned}$$

Generative Adversarial Networks: Optimality

$$\begin{aligned} & \min_{\mathbf{G}} \max_{\mathbf{D}} \left(\mathbb{E}_{x \sim p_{data}} [\log \mathbf{D}(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - \mathbf{D}(\mathbf{G}(z)) \right) \right] \right) \\ &= \min_{\mathbf{G}} \left(\mathbb{E}_{x \sim p_{data}} \left[\log \frac{p_{data}(x)}{\frac{p_{data}(x) + p_{\mathbf{G}}(x)}{2}} \right] + \mathbb{E}_{x \sim p_{\mathbf{G}}} \left[\log \frac{p_{\mathbf{G}}(x)}{\frac{p_{data}(x) + p_{\mathbf{G}}(x)}{2}} \right] - \log 4 \right) \end{aligned}$$

Generative Adversarial Networks: Optimality

$$\begin{aligned} & \min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log (1 - D(G(z)))] \right) \\ &= \min_G \left(\mathbb{E}_{x \sim p_{data}} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G} \left[\log \frac{p_G(x)}{p_{data}(x) + p_G(x)} \right] - \log 4 \right) \end{aligned}$$

Kullback-Leibler Divergence:

$$KL(p, q) = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right]$$

Generative Adversarial Networks: Optimality

$$\begin{aligned} & \min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right) \\ &= \min_G \left(\mathbb{E}_{x \sim p_{data}} \left[\log \frac{p_{data}(x)}{\frac{p_{data}(x) + p_G(x)}{2}} \right] + \mathbb{E}_{x \sim p_G} \left[\log \frac{p_G(x)}{\frac{p_{data}(x) + p_G(x)}{2}} \right] - \log 4 \right) \\ &= \min_G \left(KL \left(p_{data}, \frac{p_{data} + p_G}{2} \right) + KL \left(p_G, \frac{p_{data} + p_G}{2} \right) - \log 4 \right) \end{aligned}$$

Kullback-Leibler Divergence:

$$KL(p, q) = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right]$$

Generative Adversarial Networks: Optimality

$$\begin{aligned} & \min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right) \\ &= \min_G \left(\mathbb{E}_{x \sim p_{data}} \left[\log \frac{p_{data}(x)}{\frac{p_{data}(x) + p_G(x)}{2}} \right] + \mathbb{E}_{x \sim p_G} \left[\log \frac{p_G(x)}{\frac{p_{data}(x) + p_G(x)}{2}} \right] - \log 4 \right) \\ &= \min_G \left(KL \left(p_{data}, \frac{p_{data} + p_G}{2} \right) + KL \left(p_G, \frac{p_{data} + p_G}{2} \right) - \log 4 \right) \end{aligned}$$

Jensen-Shannon Divergence:

$$JSD(p, q) = \frac{1}{2} KL \left(p, \frac{p + q}{2} \right) + \frac{1}{2} KL \left(q, \frac{p + q}{2} \right)$$

Generative Adversarial Networks: Optimality

$$\begin{aligned} & \min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log (1 - D(G(z)))] \right) \\ &= \min_G \left(\mathbb{E}_{x \sim p_{data}} \left[\log \frac{p_{data}(x)}{\frac{p_{data}(x) + p_G(x)}{2}} \right] + \mathbb{E}_{x \sim p_G} \left[\log \frac{p_G(x)}{\frac{p_{data}(x) + p_G(x)}{2}} \right] - \log 4 \right) \\ &= \min_G \left(KL \left(p_{data}, \frac{p_{data} + p_G}{2} \right) + KL \left(p_G, \frac{p_{data} + p_G}{2} \right) - \log 4 \right) \\ &= \min_G (2 * JSD(p_{data}, p_G) - \log 4) \end{aligned}$$

JSD is always nonnegative, and zero only when the two distributions are equal!
Thus $p_{data} = p_G$ is the global min, QED

Jensen-Shannon Divergence:

$$JSD(p, q) = \frac{1}{2} KL \left(p, \frac{p + q}{2} \right) + \frac{1}{2} KL \left(q, \frac{p + q}{2} \right)$$

Generative Adversarial Networks: Optimality

$$\min_G \max_D \left(\mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} \left[\log \left(1 - D(G(z)) \right) \right] \right)$$
$$= \min_G (2 * JSD(p_{data}, p_G) - \log 4)$$

Summary: The global minimum of the minimax game happens when:

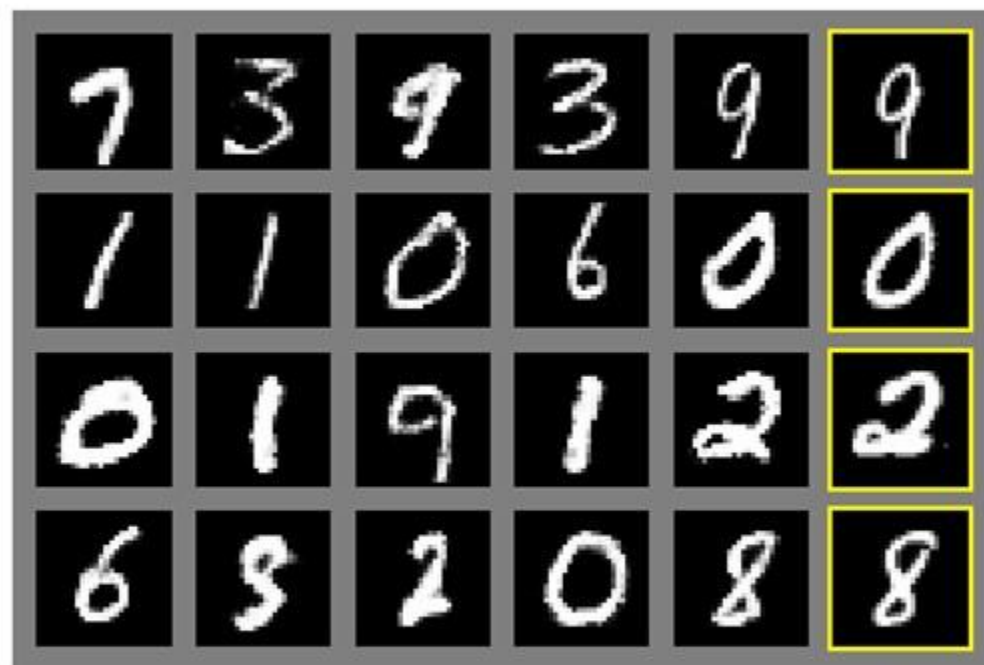
1. $D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_G(x)}$ (Optimal discriminator for any G)
2. $p_G(x) = p_{data}(x)$ (Optimal generator for optimal D)

Caveats:

1. G and D are neural nets with fixed architecture. We don't know whether they can actually represent the optimal D and G.
2. This tells us nothing about convergence to the optimal solution

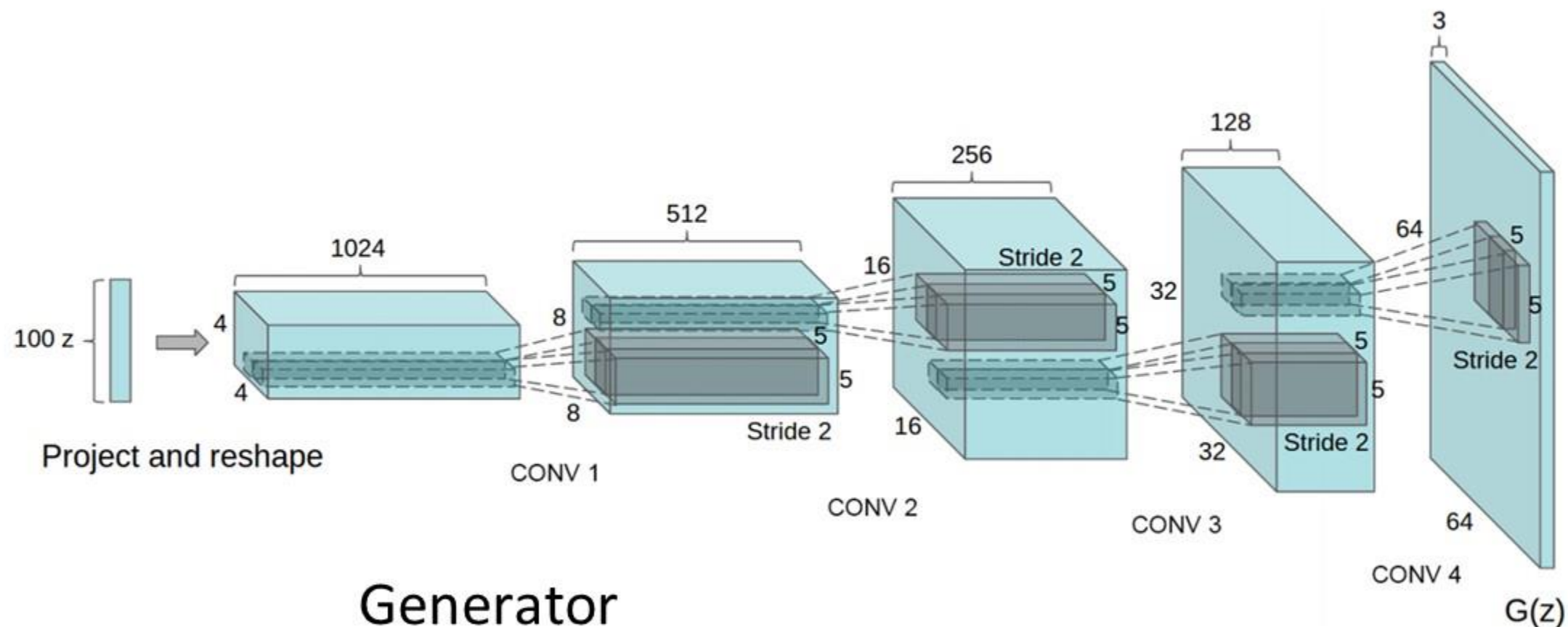
Generative Adversarial Networks: Results

Generated samples



Nearest neighbor from training set

Generative Adversarial Networks: DC-GAN



Generative Adversarial Networks: DC-GAN

Samples from the model look much better!



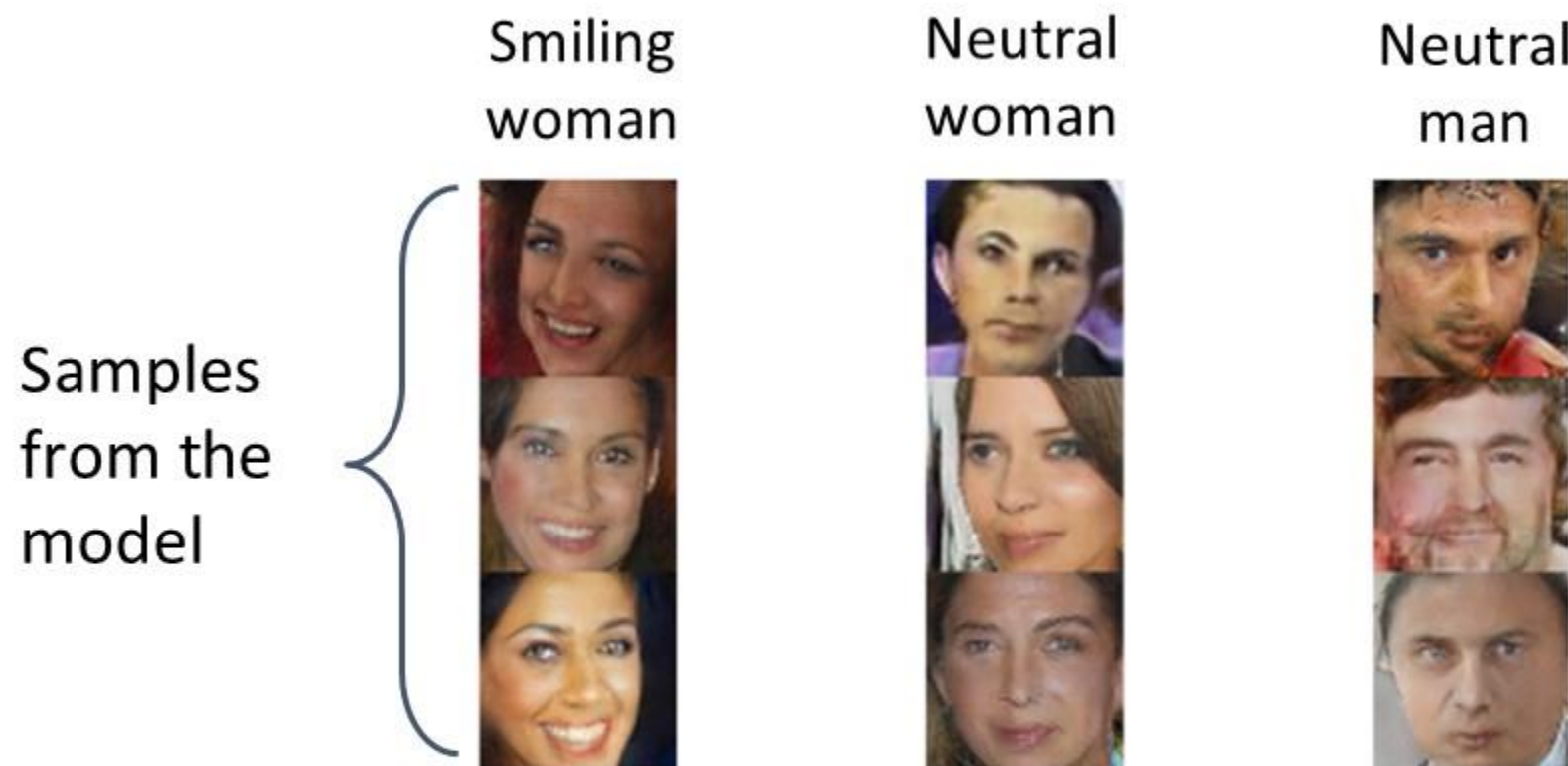
Generative Adversarial Networks: Interpolation

Interpolating
between
points in
latent z
space

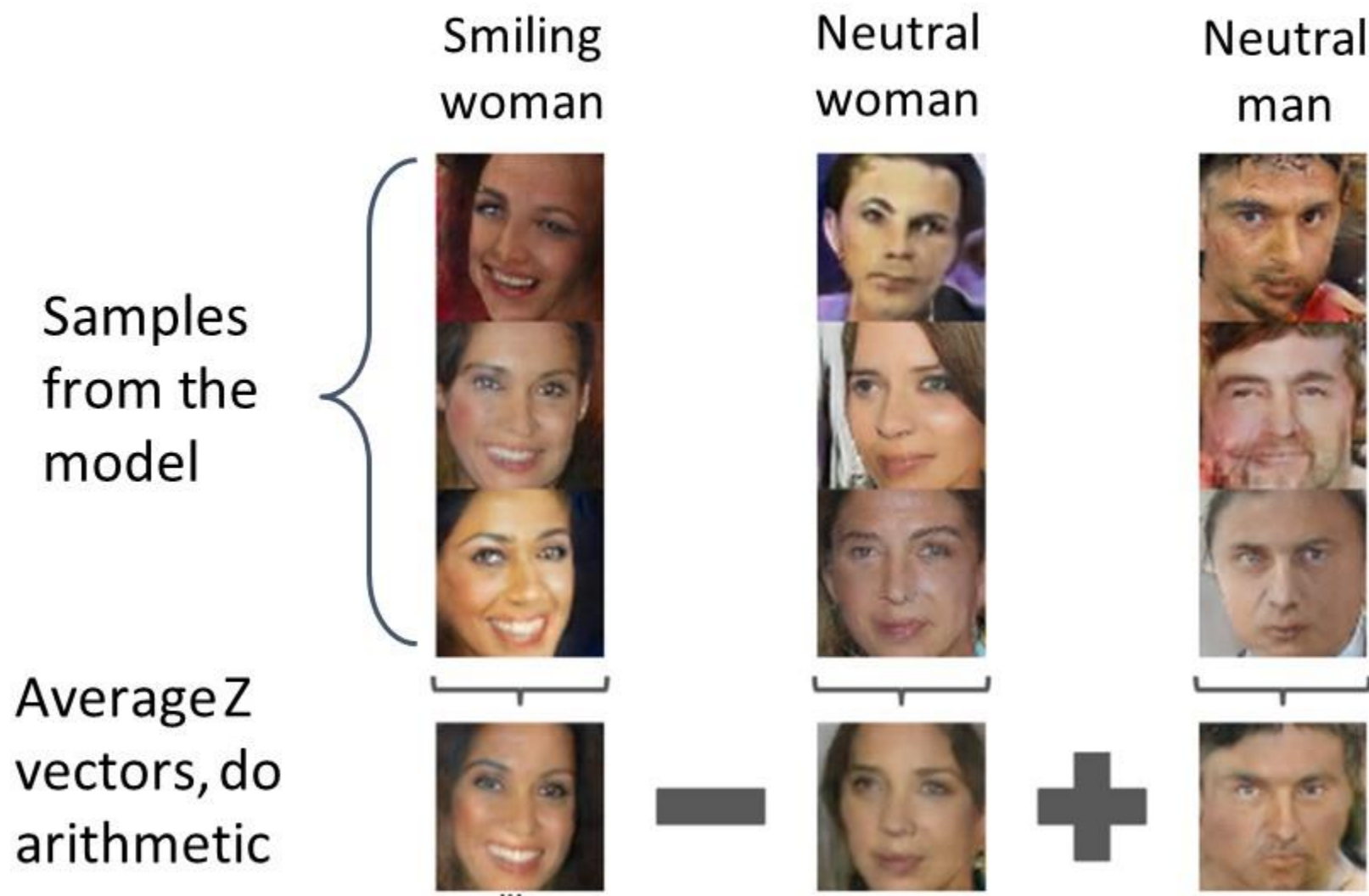


Radford et al,
ICLR 2016

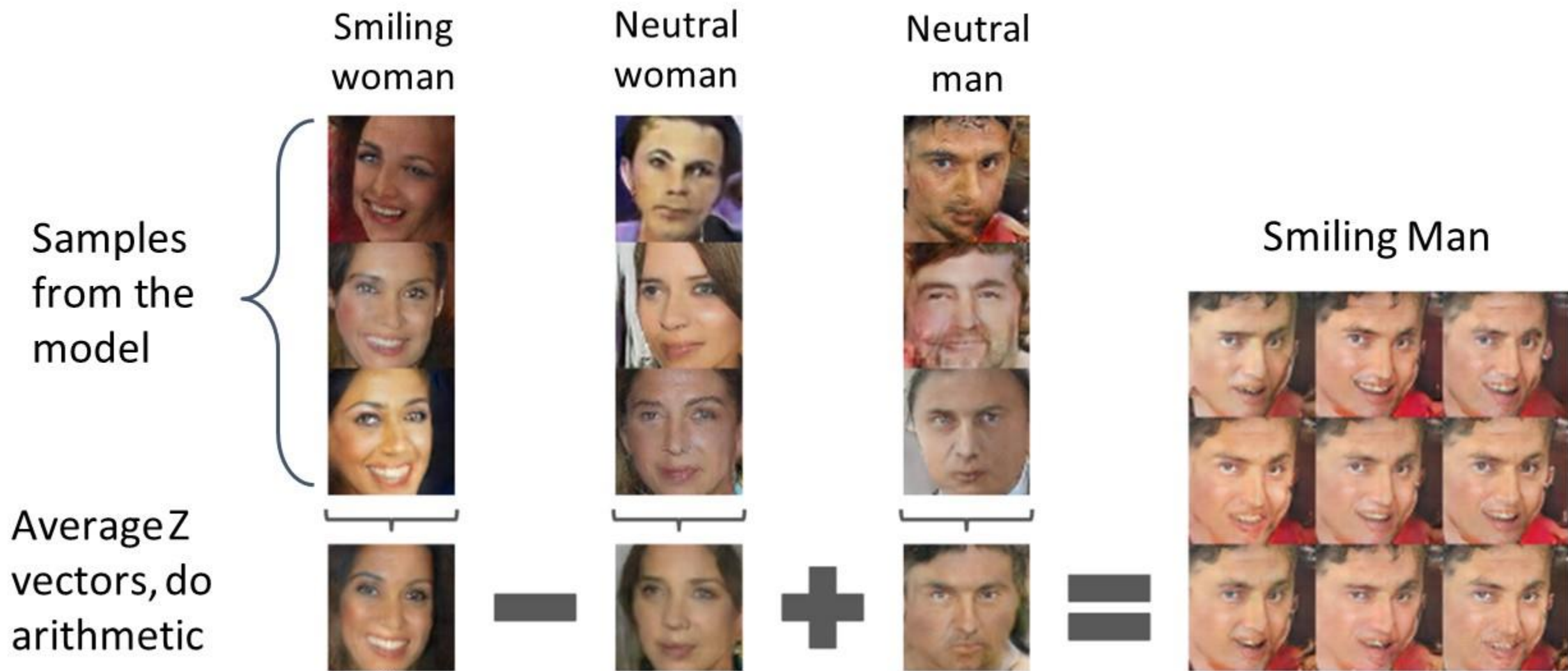
Generative Adversarial Networks: Vector Math



Generative Adversarial Networks: Vector Math



Generative Adversarial Networks: Vector Math



Generative Adversarial Networks: Vector Math

Man with
glasses

Man w/o
glasses

Woman
w/o glasses

Samples
from the
model

Average Z
vectors, do
arithmetic



Generative Adversarial Networks: Vector Math

Samples
from the
model

Man with
glasses



Average Z
vectors, do
arithmetic



Man w/o
glasses



Woman
w/o glasses



-

+

=

Woman with
glasses



GAN Improvements: Improved Loss Functions

Wasserstein GAN (WGAN)



Arjovsky, Chintala, and Bottou, "Wasserstein GAN", 2017

WGAN with Gradient Penalty (WGAN-GP)



Gulrajani et al, "Improved Training of Wasserstein GANs", NeurIPS 2017

GAN Improvements: Higher Resolution

256 x 256 bedrooms



1024 x 1024 faces



GAN Improvements: Higher Resolution

512 x 384 cars



1024 x 1024 faces



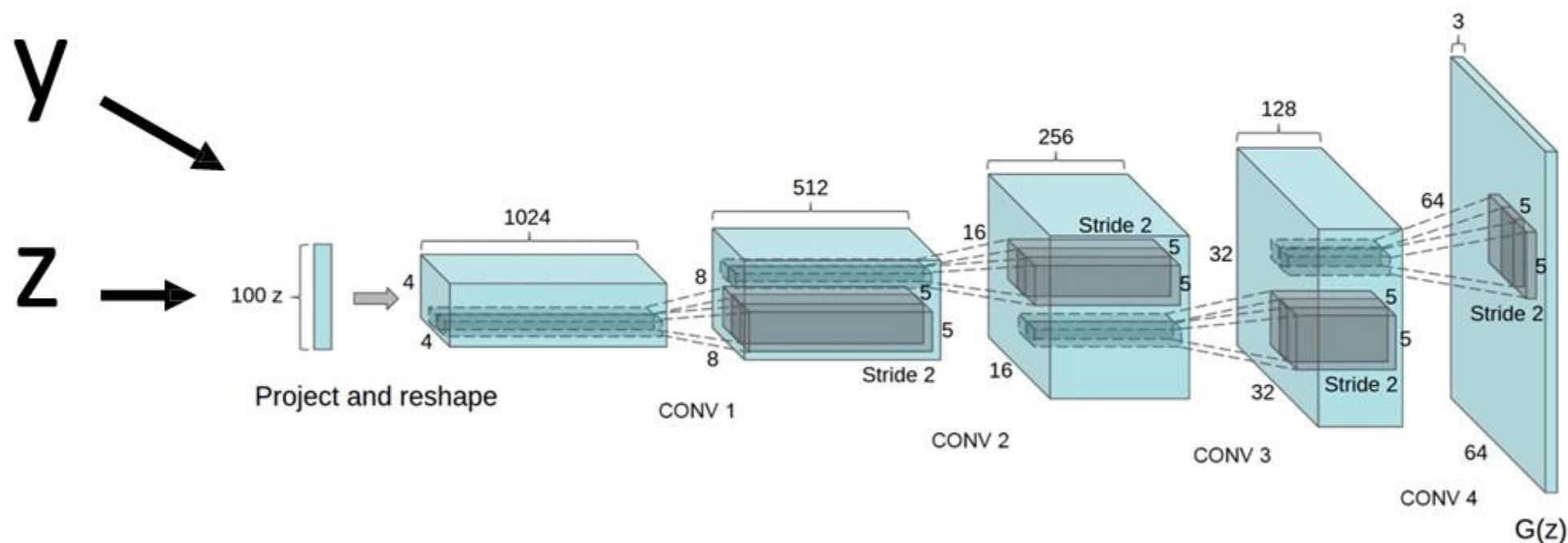
StyleGAN2



Karras et al, "Analyzing and Improving the Image Quality of StyleGAN", CVPR 2020

Conditional GANs

Recall: Conditional Generative Models learn $p(x|y)$ instead of $p(x)$
Make generator and discriminator both take label y as an additional input!



Conditional GANs: Conditional Batch Normalization

Batch Normalization

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Learn a separate
scale and shift
for each
different label y

Conditional Batch Normalization

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

$$y_{i,j} = \gamma_j^y \hat{x}_{i,j} + \beta_j^y$$

Conditional GANs: Spectral Normalization

Welsh springer spaniel



Fire truck



Daisy



Miyato et al, "Spectral Normalization for Generative Adversarial Networks", ICLR 2018

128x128 images on ImageNet

March 30, 2022

Conditional GANs: Self-Attention

Goldfish



Indigo bunting



Redshank



Saint Bernard



Conditional GANs: BigGAN



Brock et al, "Large Scale GAN Training for High Fidelity Natural Image Synthesis", ICLR 2019

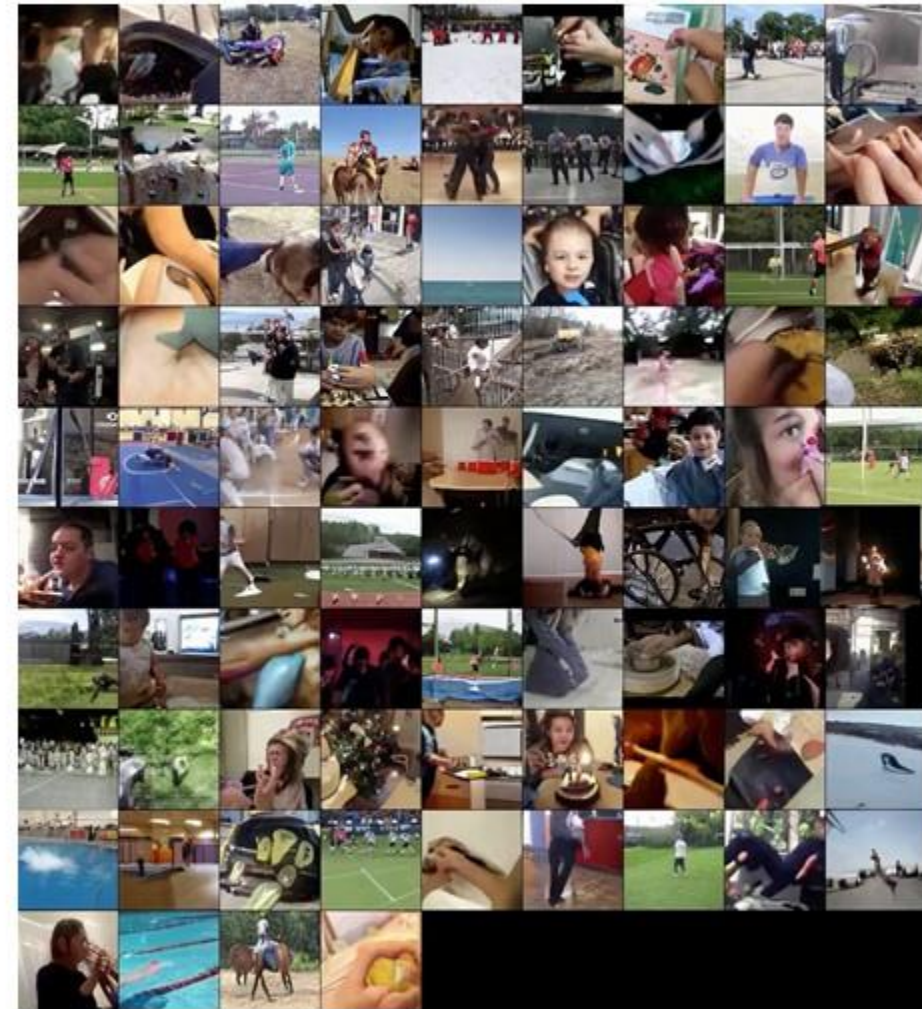
512x512 images on ImageNet

Generating Videos with GANs



64x64 images, 48 frames

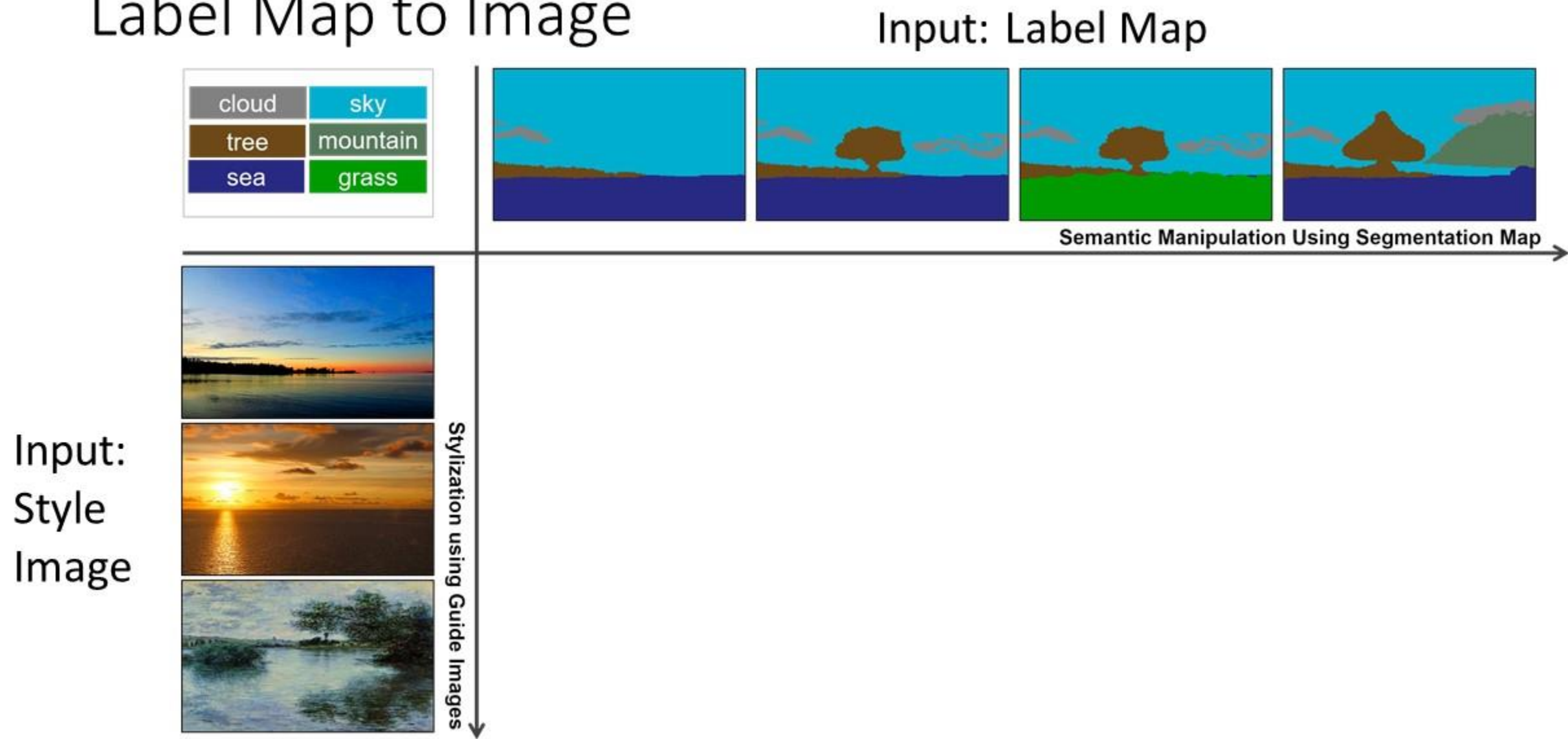
<https://drive.google.com/file/d/1FOQYdUuxPXvS8yeOhXdPQMMapUQakLi/view>



128x128 images, 12 frames

https://drive.google.com/file/d/16SYxuvvu3viQy-39LhhSDGtczbWphj_i/view

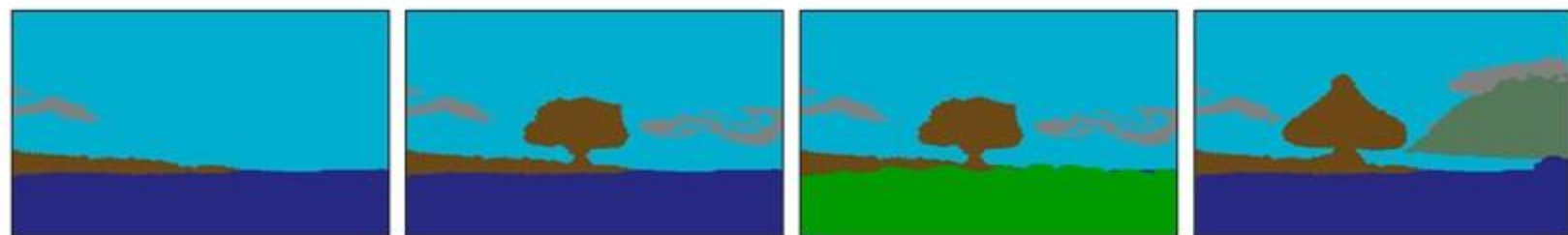
Label Map to Image



Label Map to Image

Input: Label Map

cloud	sky
tree	mountain
sea	grass



Semantic Manipulation Using Segmentation Map →

Input:
Style
Image



Stylization using Guide Images ↓



Conditioning on more than labels! Text to Image

This bird is red and brown in color, with a stubby beak



The bird is short and stubby with yellow on its body



A bird with a medium orange bill white body gray wings and webbed feet



This small black bird has a short, slightly curved bill and long legs



A picture of a very clean living room



A group of people on skis stand in the snow



Eggs fruit candy nuts and meat served on white dish



A street sign on a stoplight pole in the middle of a day



Zhang et al, "StackGAN++: Realistic Image Synthesis with Stacked Generative Adversarial Networks.", TPAMI 2018

Zhang et al, "StackGAN: Text to Photo-realistic Image Synthesis with Stacked Generative Adversarial Networks.", ICCV 2017

Reed et al, "Generative Adversarial Text-to-Image Synthesis", ICML 2016

Text to Image: DALL-E

Step 1: Train VQ-VAE (discrete grid of latent codes)

Step 2: Train autoregressive Transformer model to predict sequence of latent codes
(Giant model on 250M image/text pairs)

Step 3: Given text prompt, sample new image codes; pass through VQ-VAE decoder to generate images

Text to Image: DALL-E

Step 1: Train VQ-VAE (discrete grid of latent codes)

Step 2: Train autoregressive Transformer model to predict sequence of latent codes
(Giant model on 250M image/text pairs)

Step 3: Given text prompt, sample new image codes; pass through VQ-VAE decoder to generate images



an illustration of a baby hedgehog in a christmas sweater walking a dog

Text to Image: DALL-E

Step 1: Train VQ-VAE (discrete grid of latent codes)

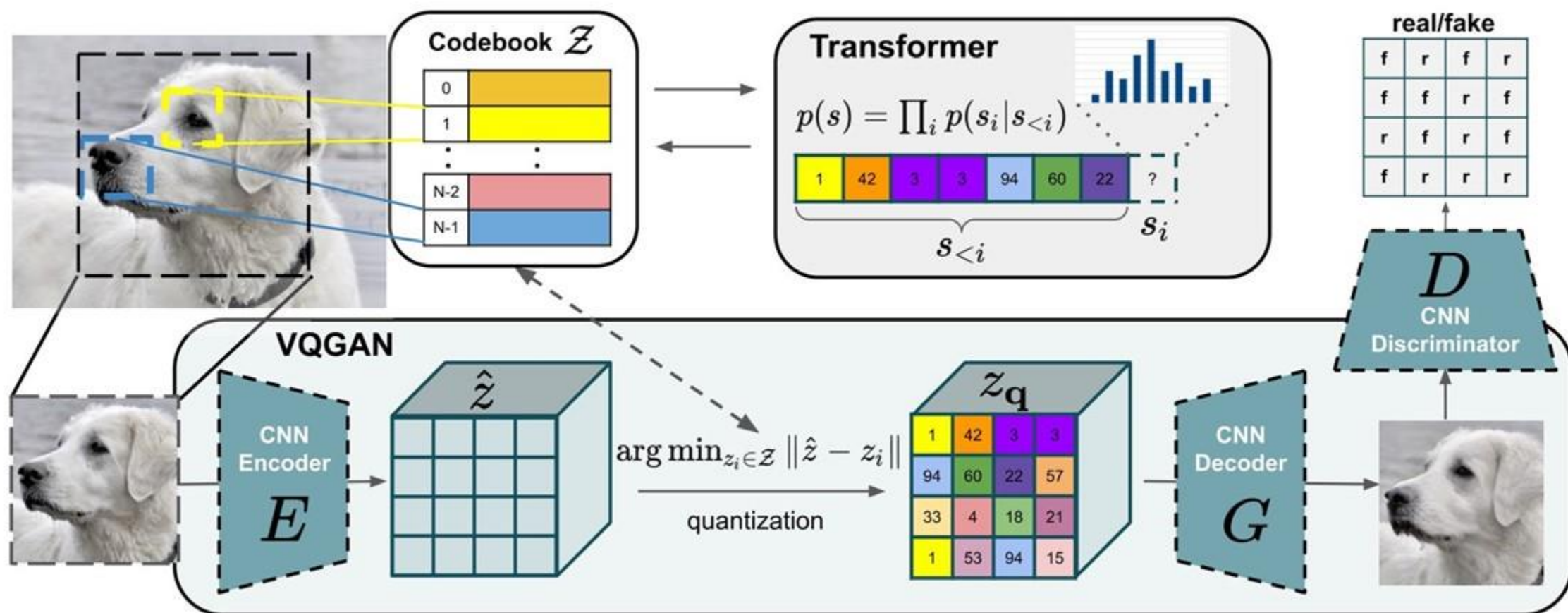
Step 2: Train autoregressive Transformer model to predict sequence of latent codes (Giant model on 250M image/text pairs)

Step 3: Given text prompt, sample new image codes; pass through VQ-VAE decoder to generate images



a neon sign that reads "backprop". a neon sign that reads "backprop". backprop neon sign

VQ-GAN



VQ-GAN (Semantic Segmentation to Image)



Image Super-Resolution: Low-Res to High-Res

bicubic
(21.59dB/0.6423)



SRResNet
(23.53dB/0.7832)



SRGAN
(21.15dB/0.6868)



original

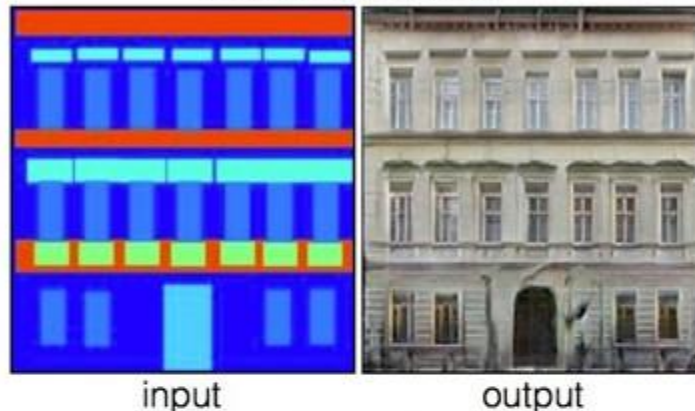


Image-to-Image Translation: Pix2Pix

Labels to Street Scene



Labels to Facade



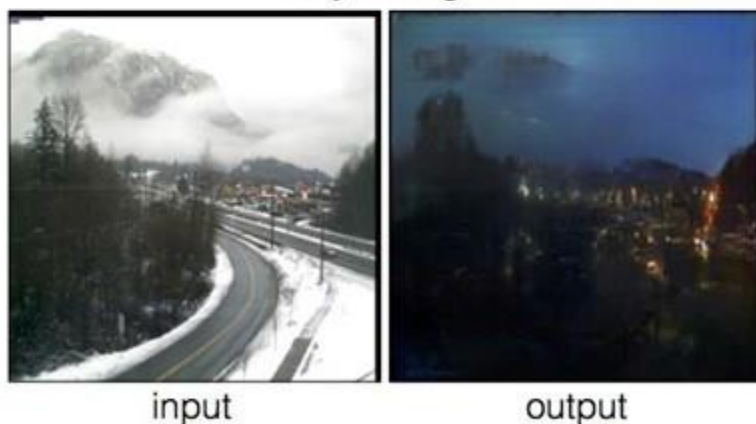
BW to Color



Aerial to Map



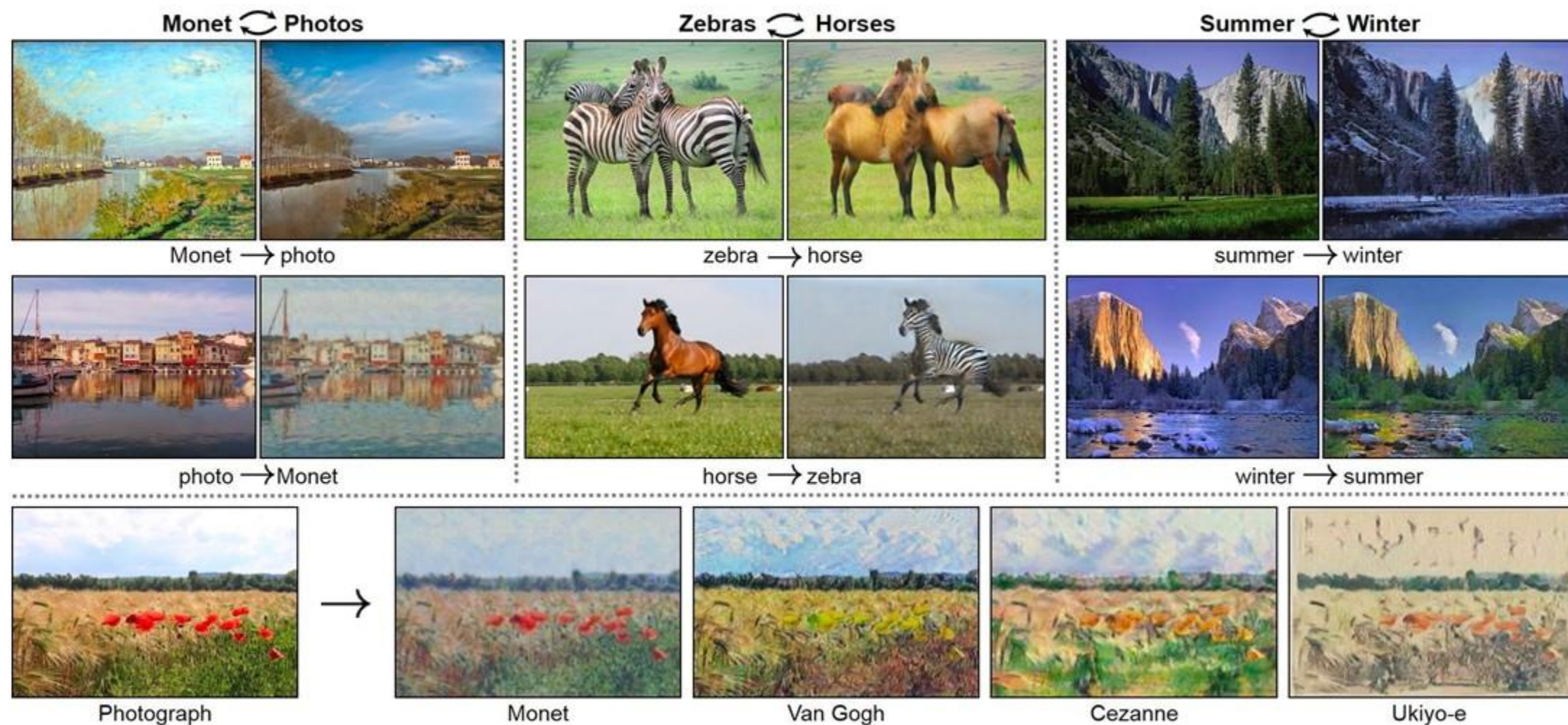
Day to Night



Edges to Photo



Unpaired Image-to-Image Translation: CycleGAN



Unpaired Image-to-Image Translation: CycleGAN

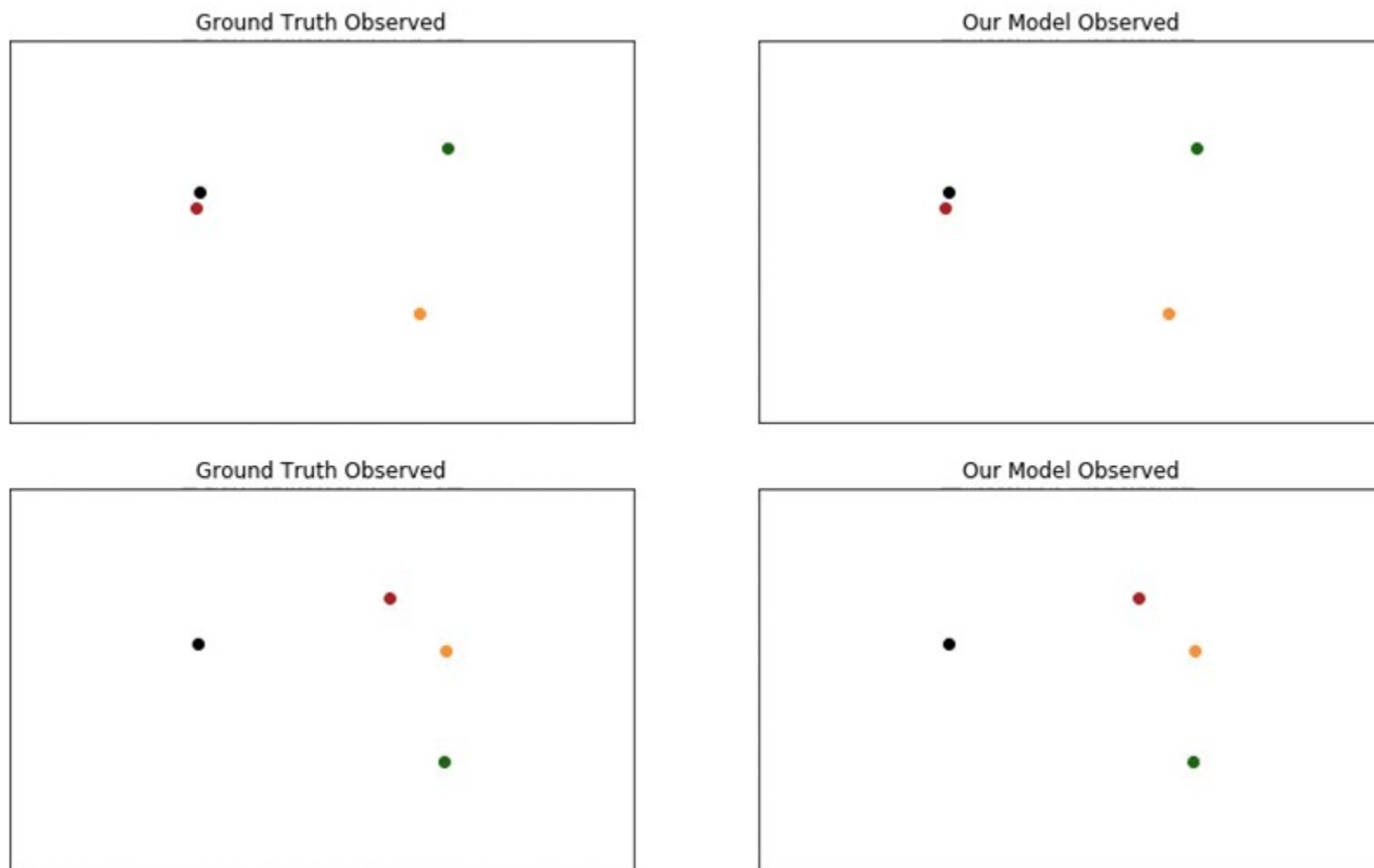
Input Video: Horse

Output Video: Zebra



<https://www.youtube.com/watch?v=9reHvktowLY>

GANs: Not just for images! Trajectory Prediction

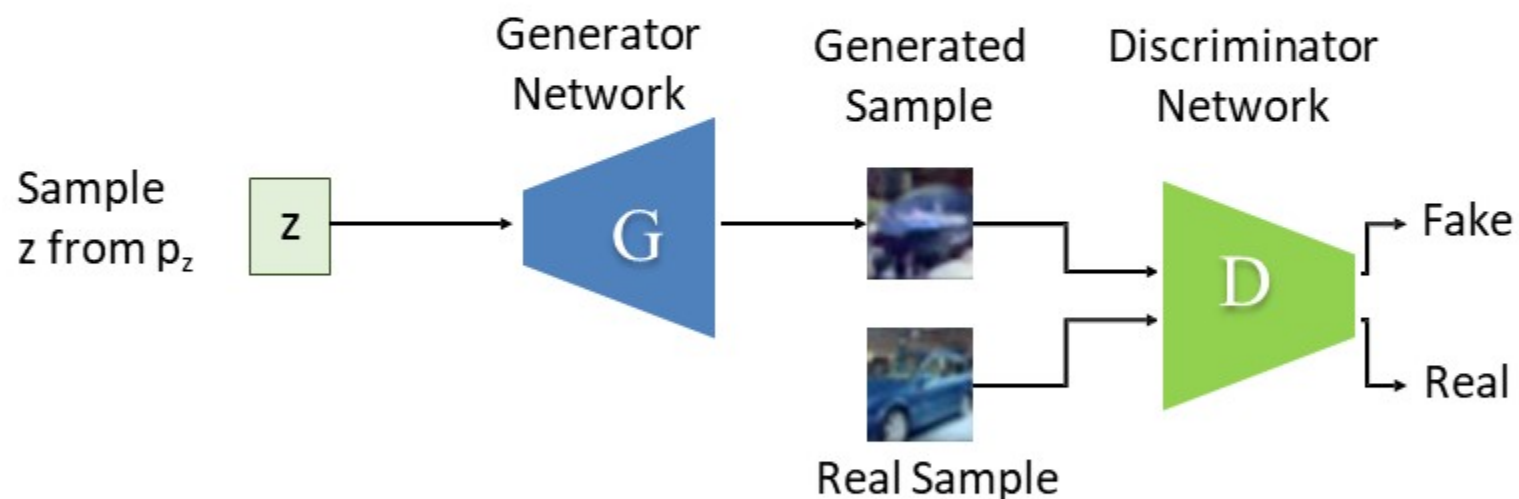


GAN Summary

Jointly train two networks:

Discriminator: Classify data as real or fake

Generator: Generate data that fools the discriminator



Under some assumptions, generator converges to true data distribution
Many applications! Very active area of research!

Generative Models Summary

Autoregressive Models directly maximize likelihood of training data:

$$p_{\theta}(x) = \prod_{i=1}^N p_{\theta}(x_i | x_1, x_2, \dots, x_{i-1})$$

Good image quality, can evaluate with perplexity. Slow to generate data, needs tricks to scale up.

Variational Autoencoders introduce a latent z , and maximize a lower bound:

$$p_{\theta}(x) = \int p_{\theta}(x|z)p(z) dz \geq \mathbb{E}_{z \sim q_{\phi}(z|x)} [\log p_{\theta}(x|z)] - D_{KL}(q_{\phi}(z|x) \| p(z))$$

Latent z allows for powerful interpolation and editing applications.

Generative Adversarial Networks give up on modeling $p(x)$, but allow us to draw samples from $p(x)$. Difficult to evaluate, but best qualitative results today